Cartel

Application web progressive (PWA) pour photographier une œuvre, l'identifier, lire son cartel, consulter une fiche sourcée, poser des questions à une IA et garder ses découvertes dans une collection personnelle.

Le nom vient du cartel de musée, l'étiquette posée à côté de l'œuvre. C'est aussi l'élément graphique central de l'interface.

Sommaire

1. État des fonctionnalités
2. Lancer l'application
3. Clés API
4. Coûts prévisibles
5. Installer sur un téléphone
6. Architecture
7. Comment fonctionne la reconnaissance
8. Tests
9. Reste à faire

État des fonctionnalités

Légende. **Testé** : implémenté et vérifié par des tests automatiques. **Implémenté, non vérifié en réel** : le code existe et ses formats sont testés sur des réponses simulées, mais il n'a pas pu être exécuté contre le service réel depuis l'environnement de développement (pas d'accès réseau aux musées, pas de clé API). **Simulé** : comportement de démonstration. **En attente** : non réalisé.

Fonction	État
Interface complète, 11 écrans, mobile, tablette, ordinateur, mode sombre	Testé (parcours navigateur)
Prise de photo (caméra du navigateur) et import d'image	Import testé ; caméra en direct à vérifier sur téléphone (HTTPS requis)
Compression avant envoi (1568 px, JPEG), refus des fichiers trop lourds, petits ou illisibles	Testé
Identification par Claude (vision + sortie JSON structurée)	Implémenté, non vérifié en réel (requêtes et réponses testées sur fetch simulé)
Vérification dans les catalogues : Joconde/POP, Wikidata, Art Institute of Chicago, Met, Rijksmuseum, Smithsonian	Implémenté, non vérifié en réel (analyse des réponses testée)
Comparaison visuelle photo / reproduction de musée	Implémenté et testé localement
Score de confiance combiné, plusieurs correspondances, statut « incertain »	Testé

Fonction	État
Lecture d'un cartel par IA, validation par l'utilisateur avant enregistrement	Implémenté, non vérifié en réel
Lecture d'un cartel sans IA (OCR Tesseract sur le téléphone)	Implémenté ; le moteur se télécharge au premier usage, non testable hors ligne ici
Association cartel / œuvre analysée juste avant	Implémenté
Fiche détaillée : faits sourcés, interprétations et hypothèses distingués, sources cliquables	Testé (fiche documentaire et fiche IA simulée)
Rétrogradation automatique d'un « fait » non sourcé par l'IA	Testé
Assistant conversationnel en flux, contexte de l'œuvre conservé, historique enregistré	Testé avec un fournisseur simulé
Collection : enregistrer, notes, tags, collections thématiques, recherche, tri, suppression	Testé
Persistance après fermeture (IndexedDB)	Testé (rechargement de page)
Export CSV et PDF	Testé
Sauvegarde et restauration (fichier JSON) pour changer d'appareil	Implémenté
Découverte au hasard (domaine public de l'AIC)	Implémenté, non vérifié en réel
Exploration par mouvement artistique	Implémenté (recherche dans les catalogues)
Carte : musées et œuvres publiques (Wikidata), mes découvertes géolocalisées	Implémenté ; fond de carte et données non vérifiés en réel
Géolocalisation facultative, désactivée par défaut, stockée sur l'appareil uniquement	Implémenté
Cache serveur et cache local des analyses, budget quotidien, limite de débit, code d'accès	Testé
Fournisseur d'IA interchangeable (Anthropic, compatible OpenAI, démo)	Anthropic et démo testés ; OpenAI non testé
Mode démonstration sans clé	Simulé, clairement signalé dans l'interface
Synchronisation automatique entre appareils (Supabase ou autre)	En attente (voir Reste à faire)
Traduction anglaise	Partielle (navigation et boutons principaux)

Lancer l'application

Prérequis : Node.js 20 ou plus récent.

```
npm install
cp .env.example .env        # puis renseignez ANTHROPIC_API_KEY (facultatif)
npm run dev                 # API sur :8787 et interface sur http://localhost:5173
```

Sans clé, l'application démarre en **mode démonstration** : bandeau visible en haut de l'écran, hypothèses d'identification fixes, fiches construites uniquement à partir des notices de musée, lecture de cartel par OCR local.

Version de production sur une seule adresse :

```
npm run build
npm start                    # http://localhost:8787 sert l'API et l'application
```

Pour tester hors ligne avec les notices de démonstration intégrées :

```
SOURCES_OFFLINE=1 AI_PROVIDER=demo npm start
```

Clés API

Votre abonnement Claude ne suffit pas

Un abonnement Claude payant (Pro, Max, Team ou Enterprise) **ne donne pas accès à l'API** et ne peut pas alimenter une application externe. L'API se paie à part, à l'usage, depuis la Console Anthropic. Source : I have a paid Claude subscription, why do I have to pay separately to use the Claude API and Console?

Ne communiquez jamais votre mot de passe Claude à une application : il ne servirait à rien ici.

Obtenir une clé Anthropic

1. Créez un compte sur la Console : https://platform.claude.com
2. Ajoutez un moyen de paiement et un petit crédit (quelques dollars suffisent pour des centaines de scans avec la configuration par défaut).
3. Fixez une limite de dépense mensuelle dans la Console.
4. Créez une clé API et placez-la dans `ANTHROPIC_API_KEY`, **côté serveur uniquement** (fichier `.env` en local, variables d'environnement de l'hébergeur en ligne). Elle n'est jamais envoyée au navigateur.

Autres clés (facultatives)

Variable	Rôle	Où l'obtenir
`SMITHSONIAN_API_KEY`	Active la source Smithsonian Open Access	Gratuite sur https://api.data.gov/signup/
`APP_ACCESS_CODE`	Code demandé par l'application ; empêche des inconnus de consommer votre crédit si l'URL circule	Choisissez-le vous-même
`OPENAI_API_KEY`	Fournisseur alternatif compatible OpenAI	Chez le fournisseur choisi

Les autres sources (Joconde, Wikidata, Art Institute of Chicago, Met, Rijksmuseum) ne demandent pas de clé. Renseignez `CONTACT_EMAIL` : Wikidata et l'AIC demandent un User-Agent identifiable.

Coûts prévisibles

Tarifs relevés le 25 octobre 2026 sur la page tarifaire de l'API Claude, en dollars par million de tokens : Sonnet 5.5 à 2 $ en entrée et 10 $ en sortie, Haiku 5.5 à 0,10 $ et 0,50 $, Opus 5.5 à 4 $ et 20 $. Une

photo de 1568 px de grand côté représente environ 1 600 tokens d'image (formule de la documentation Vision).

Estimations calculées par l'application (`server/lib/cost.ts`), réflexion du modèle comprise :

Configuration	Coût d'un scan complet (identification + fiche)
Par défaut : Sonnet 5.5 pour identifier et rédiger	0,05 à 0,12 $
Économique : Haiku 5.5 partout	0,002 à 0,006 $ (identification moins fiable)
Précision maximale : Opus 5.5 pour identifier	0,06 à 0,16 $
Une question à l'assistant (Haiku 5.5)	environ 0,001 $
Lecture d'un cartel (Haiku 5.5)	environ 0,001 $

Ce ne sont que des ordres de grandeur ; le coût réel de chaque appel s'affiche sous les résultats et dans le Profil. Les sources documentaires sont gratuites. L'hébergement peut l'être aussi (offre gratuite de Render, avec mise en veille après inactivité).

Mesures en place pour limiter la dépense :

- compression et redimensionnement sur le téléphone avant envoi ;
- refus des fichiers de plus de 4 Mo après compression (`MAX_IMAGE_BYTES`) ;
- cache des analyses par empreinte de la photo, sur le serveur et sur le téléphone : une même photo n'est jamais payée deux fois ;
- cache des fiches par œuvre et par jeu de notices ;
- fiche rédigée seulement quand l'utilisateur l'ouvre, pas pour chaque hypothèse ;
- budget quotidien global (`DAILY_BUDGET_USD`, 2 $ par défaut) et limite par adresse IP (`RATE_LIMIT_PER_MINUTE`) ;
- désactivation complète de l'IA payante avec `AI_PROVIDER=demo`, l'application restant utilisable (OCR local, fiches documentaires).

Aucun système de paiement n'est intégré. Le point d'entrée naturel pour en ajouter un plus tard est le middleware de `server/app.ts`, qui contrôle déjà l'accès et le budget.

Installer sur un téléphone

La caméra du navigateur exige une adresse **HTTPS**. Deux options :

Option 1, en ligne (recommandé). Déployez sur Render :

1. Publiez le dossier sur un dépôt GitHub.
2. Sur https://render.com, choisissez *New > Blueprint* et sélectionnez le dépôt : le fichier `render.yaml` configure tout.
3. Renseignez `ANTHROPIC_API_KEY`, `APP_ACCESS_CODE` et `CONTACT_EMAIL` dans les variables d'environnement.
4. Ouvrez l'URL `https://….onrender.com` sur le téléphone.
5. iPhone : Safari, bouton Partager, *Sur l'écran d'accueil*. Android : Chrome, menu, *Installer l'application*.

6. Dans *Profil*, saisissez le code d'accès.

Un `Dockerfile` est fourni pour tout autre hébergeur (Fly.io, Railway, un VPS).

Option 2, sur le réseau local. `npm run build && npm start`, puis exposez le port en HTTPS avec un tunnel (par exemple `npx localtunnel --port 8787` ou Cloudflare Tunnel) et ouvrez l'adresse sur le téléphone. Sans HTTPS, l'import d'image et le bouton « Ouvrir l'appareil photo » fonctionnent quand même ; seul l'aperçu caméra en direct est bloqué.

Publication sur les stores. Pas nécessaire pour l'usage personnel. Si besoin plus tard, la PWA peut être empaquetée avec Capacitor (iOS et Android) sans réécrire l'interface.

Architecture

```
cartel/
├─ src/                    Interface (React + TypeScript, Vite, PWA)
│  ├─ pages/               Les 11 écrans
│  ├─ components/ui.tsx    Cartel, vignettes, jauge de confiance, icônes
│  ├─ lib/                 API, base locale (Dexie/IndexedDB), compression, export, OCR
│  ├─ i18n/                Dictionnaires de langue
│  └─ styles/app.css       Système visuel : grille, ligne de base, couleurs
├─ server/                 API (Hono sur Node)
│  ├─ ai/                  Fournisseurs d'IA interchangeables, consignes, schémas JSON
│  ├─ sources/             Un adaptateur par base documentaire
│  ├─ pipeline/            Identification, vérification, comparaison visuelle, fiche
│  └─ lib/                 Cache, coûts, limites, comparaison de textes
├─ shared/                 Types communs client/serveur, analyse de cartel
└─ tests/                  Tests unitaires, d'intégration et navigateur
```

Pourquoi ces choix

- **PWA plutôt qu'Expo/React Native.** Une seule base de code utilisable tout de suite sur téléphone et ordinateur, installable sans passer par les stores, sans compte développeur Apple (99 $ par an). L'accès caméra, le stockage local et le mode hors ligne suffisent ici. Limite : sur iPhone, le stockage d'une PWA peut être purgé si l'application n'est pas ouverte pendant longtemps ; d'où la sauvegarde JSON.
- **Hono sur Node.** Serveur léger et standard, qui garde la clé API hors du navigateur et tourne aussi bien en local que sur un hébergeur.
- **IndexedDB plutôt que Supabase pour la phase 1.** Gratuit, sans compte, fonctionne hors ligne, aucune donnée personnelle sur un serveur. Chaque enregistrement porte `updatedAt` pour préparer une synchronisation.
- **Appels HTTP directs plutôt que SDK.** L'interface `AIProvider` (`server/ai/types.ts`) isole le fournisseur : changer de fournisseur revient à écrire une classe de 100 lignes.
- **Leaflet + OpenStreetMap** : gratuit, sans clé. Pour un usage intensif, changez de fournisseur de tuiles avec `VITE_TILE_URL`.

Séparation des responsabilités

Étape	Qui s'en charge
Observation visuelle et hypothèses	Claude (vision), sortie JSON contrainte

Étape	Qui s'en charge
Lecture du texte (cartel, inscriptions)	Claude, ou Tesseract sur le téléphone
Recherche documentaire	Adaptateurs de sources, sans IA
Vérification visuelle	Code local (`server/pipeline/visual.ts`)
Score de confiance	Code local (`server/pipeline/scoring.ts`)
Rédaction de la fiche	Claude, à partir des seules notices fournies
Réponses aux questions	Claude, avec la fiche et les notices en contexte

Comment fonctionne la reconnaissance

1. La photo est compressée sur le téléphone puis normalisée sur le serveur (orientation, 1568 px).
2. Claude décrit ce qu'il voit, transcrit le texte visible et propose de zéro à trois œuvres précises avec une confiance et les indices qui les départagent. La consigne l'oblige à laisser la liste vide s'il ne reconnaît rien et à ne jamais identifier de personne.
3. Chaque hypothèse est cherchée en parallèle dans Joconde, Wikidata, l'Art Institute of Chicago, le Met, le Rijksmuseum et le Smithsonian. Une notice n'est retenue que si le titre **et** l'artiste concordent.
4. Quand une notice a une image, elle est comparée à la photo (structure en niveaux de gris sur plusieurs recadrages, plus histogramme de couleurs).
5. La confiance finale combine l'estimation du modèle (45 %), la concordance texte (30 %) et la ressemblance d'image (25 %). Sans notice concordante, elle est plafonnée à 55 % et l'œuvre est marquée **non vérifiée**. Une œuvre n'est déclarée « identifiée » que si elle est vérifiée.
6. La fiche est rédigée à partir des notices numérotées [S1], [S2]… Chaque phrase est typée *fait*, *interprétation* ou *hypothèse*. Un « fait » qui ne cite aucune notice existante est automatiquement rétrogradé et affiché comme **non sourcé**. La bibliographie n'est jamais écrite par l'IA : elle provient des notices.

Limites connues : une œuvre absente des six bases ne sera jamais « vérifiée », même correctement reconnue ; la plupart des œuvres d'art urbain, du design récent et des collections privées sont dans ce cas. La comparaison d'images reste rudimentaire face aux photos de biais ou avec reflets ; c'est un signal d'appoint.

Conditions d'utilisation des sources

Source	Accès	Licence des données	Images
Joconde (extrait open data)	API Opendatasoft, sans clé	Licence Ouverte 2.0, mise à jour hebdomadaire	Absentes de l'extrait ; lien vers la notice POP
Wikidata	API publique, User-Agent identifiable requis	CC0	Wikimedia Commons, licence propre à chaque fichier
Art Institute of Chicago	API publique, en-tête `AIC-User-Agent` demandé	CC0	Affichées seulement si `is_public_domain`
Met	API publique, sans clé	Open Access CC0	Affichées seulement si `isPublicDomain`
Rijksmuseum	API Linked Art, sans clé	Données ouvertes	Pas encore intégrées

Source	Accès	Licence des données	Images
Smithsonian	Clé api.data.gov	CC0 pour l'Open Access	Affichées seulement si CC0

Tests

```
        npm run typecheck     # vérification TypeScript
        npm test              # 45 tests unitaires et d'intégration (Vitest)
        npm run test:e2e      # 3 parcours complets dans Chromium (Playwright, mode démo
hors ligne)
```

Ce qui est couvert : comparaison de textes et faux positifs (même titre, autre artiste), calcul de confiance et statuts, garde-fous sur la sortie du modèle, analyse des réponses de chaque source, forme exacte des requêtes envoyées à l'API Claude (sortie structurée, absence de `temperature` et de `tool_choice` refusés par Sonnet 5.5), gestion des erreurs (clé invalide, quota, crédit épuisé, refus), lecture du flux SSE, API HTTP de bout en bout (identification, cache, fiche, repli sans IA, conversation, cartel, recherche), code d'accès, limite de débit, budget, absence de clé dans les réponses, analyse heuristique de cartel, export CSV ; et dans le navigateur : scan, résultats, fiche, ajout à la collection, question, persistance après rechargement, recherche, filtres, notes, tags, collection thématique, export CSV et PDF.

Les tests navigateur ont besoin de Chromium : `npx playwright install chromium`, ou `PW_CHROMIUM=/chemin/vers/chrome npm run test:e2e`.

Premier test réel à faire avec votre clé. Lancez l'application, photographiez une reproduction connue (par exemple une carte postale) puis une œuvre moins célèbre, et vérifiez dans *Profil* le coût affiché. C'est la seule étape que je n'ai pas pu exécuter.

Reste à faire

Phase 2

- Synchronisation automatique entre appareils. Proposition : Supabase (authentification par lien magique, table `artworks` en JSON, stockage des photos), fusion par `updatedAt` comme le fait déjà l'import de sauvegarde. En attendant, la sauvegarde JSON permet de transférer une collection.
- Images du Rijksmuseum (résolution IIIF).
- Recherche d'images inversée (par exemple via une API de recherche visuelle) pour les œuvres que le modèle ne reconnaît pas.
- Utilisation de l'outil de recherche web de Claude pour les œuvres absentes des bases (10 $ pour 1 000 recherches en plus des tokens).

Phase 3

- Index d'empreintes visuelles (embeddings) des collections ouvertes pour une reconnaissance sans IA générative.
- Traduction anglaise complète, puis autres langues (un fichier par langue dans `src/i18n`).

- Tests sur téléphones réels (iPhone et Android), mesures de précision sur un jeu de photos annotées.
- Empaquetage Capacitor pour les stores.